

Xcelium™ Integrated Coverage

Week 13 Lab Document

Lab Manual

Revision 1.0

© 1990-2024 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;

- The publication may not be modified in any way;

- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and

- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Table of Contents

Xcelium Integrated Coverage

Lab 1	Identifying Coverage Types	5
	Understanding the Lab Model.....	5
	Proposing a Coverage Plan	7
Lab 2	Generating and Analyzing Code Coverage.....	9
	Identifying Block Coverage	9
	Identifying Expression Coverage	13

Lab 1 Identifying Coverage Types

Objective: To determine the coverage types to apply to your project.

Your prerequisite for this lab is that you are familiar with digital verification.

Your objective for this lab is to identify the coverage types to apply to your project.

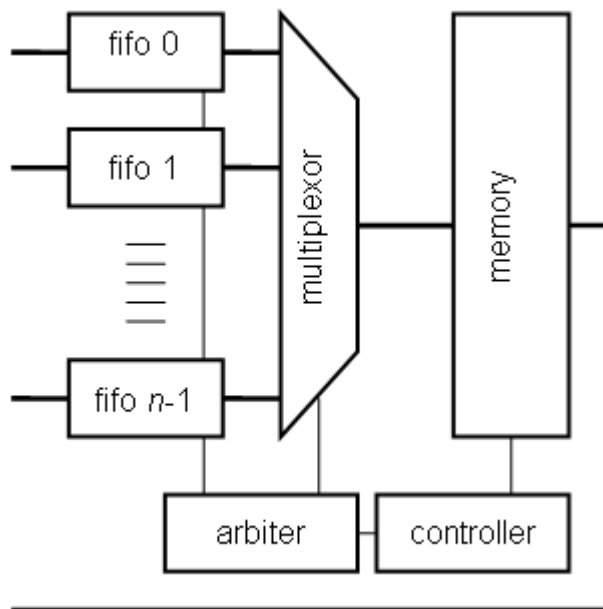
This lab focuses on identifying coverage. You will generate a coverage plan specifying what you will cover and the order in which you will apply and analyze the coverage.

This lab is an intellectual exercise. You do not run tools for this lab.

Understanding the Lab Model

Here, you familiarize yourself (at the system level) with the Memory Interface Controller.

1. Locate the Memory Interface Controller code in the `/sources/src/mic/` directory. Examine the code sufficiently to ensure that for each design unit, you have a fundamental understanding of its operation.



The Memory Interface Controller (mic) accepts read and write requests on a parameterizable number of input ports, allocates the memory resource to these requests according to an arbitration policy, and for a read request, returns the read memory data.

- The *fifo* unit is a generic synchronous FIFO. A write pointer and a read pointer chase each other in a circular fashion. When the pointers are equal, the FIFO is either full or empty. It is full if the most recent operation was to write and empty if the most recent operation was to read.
- The *arbiter* unit monitors the FIFO empty flags. A false, empty flag is a request. The arbiter grants the highest priority request and simultaneously gives that channel the lowest priority and increments the priority of all other channels.

Identifying Coverage Types

- The *multiplexor* unit utilizes the arbiter grant to select the channel to access the memory resource.
- The *controller* unit generates the memory, write and read signals, address, and data, from the selected request and the arbiter grant.
- The *memory* unit is synchronously written and asynchronously read.

Proposing a Coverage Plan

Here, you propose a coverage plan for the Memory Interface Controller.

- 1. Examine the Memory Interface Controller code in the *src/* directory under your lab root. For each design unit, determine the coverage types you would apply, the constructs to which you would apply them, and the order in which you would apply them. Consider such issues as:
 - At what abstraction level {code, signal, transaction} will I collect coverage?
 - In what domain {control, data} will I collect coverage?
 - Will I collect coverage dependently or independently of a clock?

You can enter your thoughts in this table if you write very small!

	Code	Data	Control	FSM
Arbiter				
Controller				
FIFO				
Memory				
Multiplexor				



Lab 2 Generating and Analyzing Code Coverage

Objective: To determine the code coverage types to apply to your project.

Your prerequisite for this lab is that you have determined that your verification project should include code coverage.

Your objective for this lab is to identify the design source to which code coverage applies and generate and analyze code coverage data.

This lab focuses on selecting code coverage. You will generate and analyze various types of code coverage.

Do this lab in the `lab2` directory.

Identifying Block Coverage

Here you identify the items to which you want to apply block coverage.

1. Examine the Memory Interface Controller source code located in directory `/sources/src/mic/`. Note those source code lines on which you think that a new block would begin. *If you comment on the source code file, be sure to not modify the code!*

2. Generate block coverage data; enter:

Important: This document may show the invocation line wrapped to multiple lines. Enter all such invocation lines as *one line* or escape the new line character.

```
xrun -f xrun_args.txt -covdut mic -coverage block
```

For this lab, do not concern yourself too much with the arguments and options. In a later training lab, you explore invocation line arguments and options.

You should see a "TEST PASSED" indication.

```
TEST PASSED
Simulation complete via $finish(1) at time 1578 NS + 1
../sources/src/mic/sv/mic_test.sv:106    $finish;
xcelium> exit

coverage setup:
  workdir   : ./cov_work
  dutinst   : top(top)
  scope     : scope
  testname  : test

coverage files:
  model(design data) : ./cov_work/scope/icc_5bc06e10_00000000.ucm
  data              : ./cov_work/scope/test/icc_5bc06e10_00000000.ucd
T00L:  xrun      24.03-s001: Exiting on Apr 15, 2024 at 14:45:42 IST (total: 00:00:02)
```

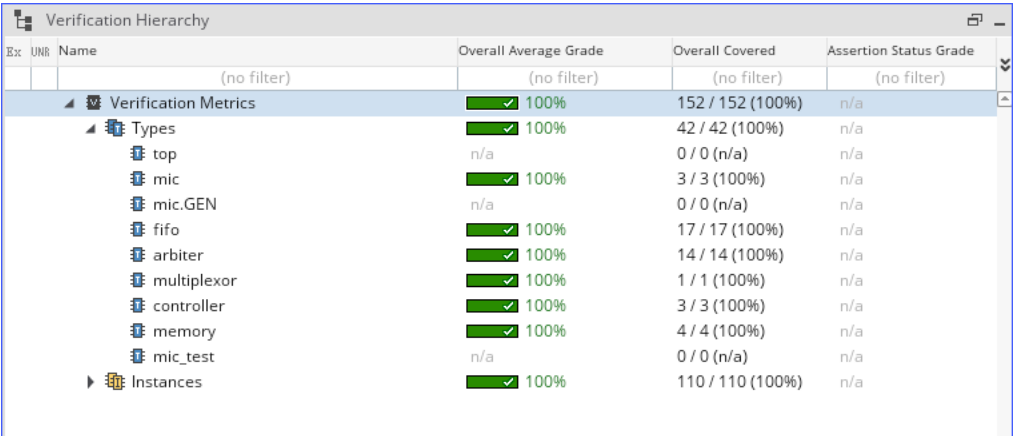
3. Examine block coverage data:

a. Enter:

```
imc -load test &
```

For this lab, do not concern yourself too much with the Integrated Metrics Center (IMC) features. In later training labs, you thoroughly explore the IMC.

b. In the Metrics Page *Verification Hierarchy* pane, expand the *Types* hierarchy and enter below the overall covered and total items (blocks).



Ex	UNR	Name	Overall Average Grade	Overall Covered	Assertion Status Grade
		(no filter)	(no filter)	(no filter)	(no filter)
		Verification Metrics	100%	152 / 152 (100%)	n/a
		Types	100%	42 / 42 (100%)	n/a
		top	n/a	0 / 0 (n/a)	n/a
		mic	100%	3 / 3 (100%)	n/a
		mic.GEN	n/a	0 / 0 (n/a)	n/a
		fifo	100%	17 / 17 (100%)	n/a
		arbiter	100%	14 / 14 (100%)	n/a
		multiplexor	100%	1 / 1 (100%)	n/a
		controller	100%	3 / 3 (100%)	n/a
		memory	100%	4 / 4 (100%)	n/a
		mic_test	n/a	0 / 0 (n/a)	n/a
		Instances	100%	110 / 110 (100%)	n/a

mic	fifo	arbiter	multiplexor	controller	memory

c. In the Metrics Page, the *Verification Hierarchy* pane navigates to and select either *Types – fifo* or *Instances – GEN[0] – fifo* and then select either from the Analysis menu the *Block Analysis* item or from the Analyze group the **Block** button.

All_Metrics

Load Data

Context info

Source Map

Block

Expression

Toggle

Statement

FSM

Cover Group

Assertion

Set Focus

Unset Focus

Hide non-focuser

Views

Context

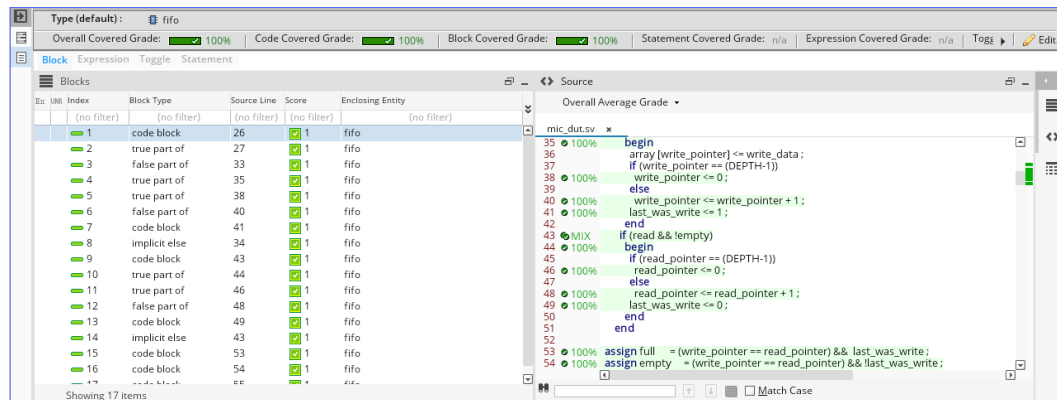
Analyze

Focus

Verification Hierarchy

Ex	UNR	Name	Overall Average Grade	Overall Covered	Assertion Status Grade
		(no filter)	(no filter)	(no filter)	(no filter)
		Verification Metrics	100%	152 / 152 (100%)	n/a
		Types	100%	42 / 42 (100%)	n/a
		top	n/a	0 / 0 (n/a)	n/a
		mic	100%	3 / 3 (100%)	n/a
		mic.GEN	n/a	0 / 0 (n/a)	n/a
		fifo	100%	17 / 17 (100%)	n/a
		arbiter	100%	14 / 14 (100%)	n/a
		multiplexor	100%	1 / 1 (100%)	n/a
		controller	100%	3 / 3 (100%)	n/a
		memory	100%	4 / 4 (100%)	n/a
		mic_test	n/a	0 / 0 (n/a)	n/a
		Instances	100%	110 / 110 (100%)	n/a

- d. In the fifo Code Page *Source* pane, confirm (or not) the code lines you expected to be the beginning of new blocks.



Are any blocks not scored? Are any scored blocks not covered? If so, what functionality does this imply that the test fails to exercise?

- e. Repeat the previous two steps for the arbiter type (or instance).
- f. Repeat the previous step for the memory type (or instance).

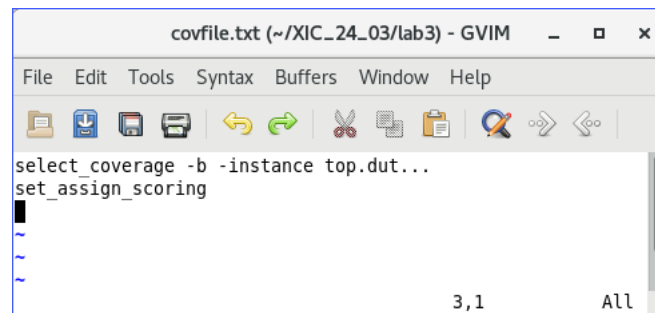
Why do the multiplexor, controller, and Memory Interface Controller (mic) not have scored blocks?

Answer: These design units have no procedural blocks to score!

4. Score non-procedural blocks:

- a. Create a text file of any name (for example, *covfile.txt*) and into it enter these coverage configuration commands:

```
select_coverage -b -instance top.dut...
set_assign_scoring
```



- b. Regenerate block coverage data; enter:

Important: Be sure to enter the name of *your* coverage configuration file!

```
xrun -f xrun_args.txt -covfile covfile.txt
```

- c. With the IMC **Load Data** button select to reload *cov_test/scope/test*.

- d. Enter below the overall covered and total items.

mic	fifo	arbiter	multiplexor	controller	memory

- e. Using what you previously learned, examine the source code to locate the additional scored blocks (the continuous assignments).

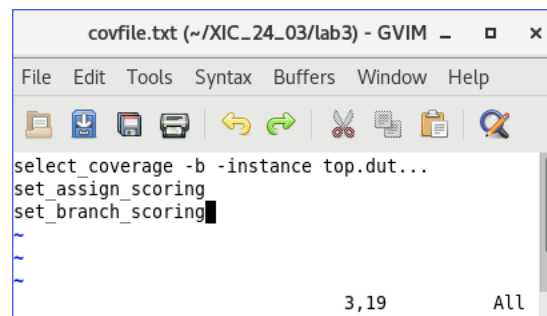
Are any continuous assignments not scored? Are any scored continuous assignments not covered? If so, what functionality does this imply that the test fails to exercise?

5. Add block branch scoring:

For most procedural blocks, block branches are themselves blocks, so scoring branches is mostly redundant. Two situations where branch scoring adds useful information are when multiple Verilog case statement matches execute the same block and for Verilog ternary expressions and VHDL conditional and selected signal assignments.

- a. Add to your coverage configuration file the command:

```
set_branch_scoring
```



- b. Regenerate block coverage data as previously.
- c. In the IMC, reload the coverage data, and enter below the overall covered and total items.

mic	fifo	arbiter	multiplexor	controller	memory

- d. Examine the source code to locate the additional scored branches (of the ternary expressions).

Are any branches not scored? Are any scored branches not covered? If so, what functionality does this imply that the test fails to exercise?

Hint: In the Code Page *Blocks* pane, you can select the **Branches Only** radio button to filter the display to only the branches.

Identifying Expression Coverage

Here you identify the items to which you want to apply expression coverage.

1. Examine the Memory Interface Controller source code. Note those expressions that you think would, by default, be covered. *If you comment on the source code file, be sure to not modify the code!*
2. Generate expression coverage data *without* a coverage configuration file by invoking the simulator as you first did but with the “block” coverage argument changed to “expr”.

```
xrun -f xrun_args.txt -covdut mic -coverage expr
```

3. Examine expression coverage data:
 - a. Enter below the overall covered and total items (expression terms).

mic	fifo	arbiter	multiplexor	controller	memory

- b. In the Metrics Page, *Verification Hierarchy* pane, navigate to and select a type or instance having scored expression terms and then select either from the Analysis menu the *Expression Analysis* item or from the Analyze group the **Expression** button.
- c. In the Code Page *Source* pane, confirm (or not) the expressions you expected to be scored.

Are any expressions not scored? Are any scored expressions not fully covered? What term values never occurred? If so, what functionality does this imply that the test fails to exercise?

Hint: Select a Top level Expression and select a Coverage Table (if more than one), then if needed, drag the *Coverage Table* pane sash to fully reveal the SOP table. Resting the cursor over a term pops up the term expression.

- d. Repeat the previous two steps for the other types or instances having scored expression terms.

Why do the multiplexor, controller, memory, and Memory Interface Controller (mic) not have scored expressions?

Answer: The Integrated Coverage scores expressions by default of only procedural controls having logical operators.

4. Score additional operators:

- a. Modify your coverage configuration file to select coverage “-e” instead of “-b” and add the below command.

```
set_expr_coverable_operators -all
```

- b. Regenerate expression coverage data *with* the coverage configuration file as you did before to regenerate block coverage data.
- c. Reload the coverage data and enter below the overall covered and total items.

mic	fifo	arbiter	multiplexor	controller	memory

- d. Examine the source code to locate the additional scored expressions.

What additional operations are now scored?

Answer: You should now see scoring include relational, reduction, and bitwise operations in procedural control path code. This configuration also scores other operators not present in this code.

Are any of the additional scored expressions not covered? If so, what functionality does this imply that the test fails to exercise?

5. Score additional statements:

- a. Modify your coverage configuration file to add the below command.

```
set_expr_coverable_statements -all
```

- b. Regenerate expression coverage data *with* the coverage configuration file.
- c. Reload the coverage data and enter below the overall covered and total items.

mic	fifo	arbiter	multiplexor	controller	memory

- d. Examine the source code to locate the additional scored statements.

What additional statements are now scored?

Answer: You should now see scoring include scored operators in continuous assignment statements. This configuration also scores other statements not present in this code.

Are any of the additional scored statements not covered? If so, what functionality does this imply that the test fails to exercise?

- e. Exit the IMC.



SystemVerilog for Design and Verification

Engineer Explorer Series

Week 13 Lab Document

Lab Manual

Revision 1.0

© 1990-2024 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

The publication may be used solely for personal, informational, and noncommercial purposes;

The publication may not be modified in any way;

Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and

Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Table of Contents

SystemVerilog for Design and Verification

Overview of Labs..... 21

Software Releases 21

Lab 1 **Simulating Simple Implication Assertions..... 23**

 Creating the Assertions 24

 Simulating Your Assertions 24

 Modifying the Assertions..... 25

 Conclusions..... 25

Lab 2 **Sequence-Based Properties 27**

 Creating the Simple Sequence Assertion 28

 Creating the Complex Sequence Assertion..... 29

 Answers to Questions..... 30

Overview of Labs

In these labs, you use SystemVerilog language constructs to complete simple, common design tasks. You can use Verilog-2001 constructs to complete some of the design tasks described here, but that would obviously defeat the purpose of the labs. Where possible, try to use SystemVerilog constructs.

This lab book assumes you are familiar with the Cadence[®] simulator, which you use in this course. If this is not the case, please ask your instructor or contact Cadence for information on running the simulator.

The goal is not to complete all the exercises during the course but to learn from each exercise at your own pace.

Software Releases

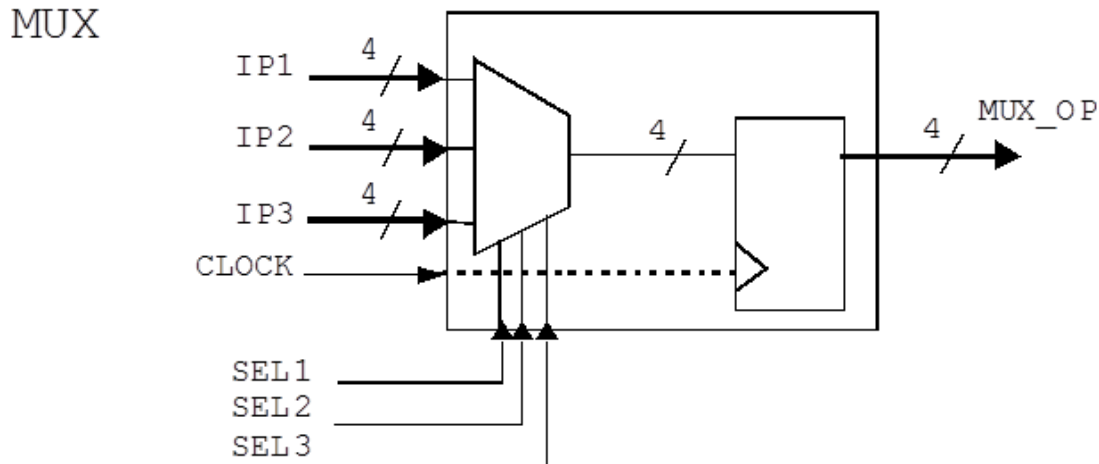
These exercises and code examples have been written and tested on the following releases:

- ◆ Xcelium[™] 24.03

Lab 1 Simulating Simple Implication Assertions

Objective: To create and simulate simple implication SystemVerilog assertions.

Write SystemVerilog assertions for an existing design to match an initial specification, then explore assertion failures and rewrite your assertions to more closely match the specification.



Specification

- ◆ `ip1`, `ip2`, `ip3` and `mux_op` are 4-bit vectors.
- ◆ Inputs are sampled on the rising edge of `clock`.
- ◆ If `sel1` is logic 1, input `ip1` is assigned to `mux_op`.
- ◆ Otherwise, if `sel2` is logic 1, input `ip2` is assigned to `mux_op`.
- ◆ Otherwise, if `sel3` is logic 1, input `ip3` is assigned to `mux_op`.

Creating the Assertions

Working in the `lab02-sv mux` directory, perform the following.

1. A model for the MUX is provided in the `mux.sv` file. In this file, assert three properties, evaluated only on the active edge (rising edge) of the signal `clock`, to do the following:
 - a. When `sel1` is 1'b1, check that in the next evaluation cycle `mux_op` equals `ip1` from the past cycle" (i.e. use `$past()`).
 - b. When `sel2` is 1'b1, check that in the next evaluation cycle `mux_op` equals `ip2` from the past cycle" (i.e. use `$past()`).
 - c. When `sel3` is 1'b1, check that in the next evaluation cycle `mux_op` equals `ip3` from the past cycle" (i.e. use `$past()`).

At this stage, **please write the properties to meet the specification** above (what the lab book asks for), rather than how you might think the properties should be defined.

Simulating Your Assertions

2. Examine the MUX testbench file, `mux_test.sv`. The first part of the testbench floats a logic 1 across each of the MUX select inputs. The second part of the testbench increments the MUX select inputs through all possible combinations.
1. Simulate your design in GUI mode using the following `xrun` options:

```
xrun mux.sv mux_test.sv -gui -access +rwc -linedebug
```
2. Before running simulation, set a simulation breakpoint on the first executable line after this comment in the testbench `mux_test.sv`:

```
//Set a breakpoint on next executable line.
```
3. Find and open the Assertion Browser window and check whether your assertions appear. Use the browser to add your assertions to the waveform window.
4. Use the Design Browser to add the assertion testbench signals to the waveform window.
5. Simulate with the first set of test vectors by running up to the breakpoint.

6. Verify that your asserted properties hold for all stimuli applied up to the breakpoint. Assertion failures will be reported by the simulator and can be viewed in the Waveform window.

There are several ways to view assertions in the waveform window, including displaying assertions as transactions. Ask your trainer or consult tool documentation.

7. Run the simulator from the breakpoint until the end of simulation.

Note that the simulator may stop every time a property fails or a breakpoint is hit. Keep running until the end of simulation as indicated by a message in the command window.

8. Your second and third property assertions have failed. Why?

Hint: Look at the properties in the waveform viewer and check when the properties fail.

Modifying the Assertions

The property specification provided above is incomplete. The properties did not consider the priority of the `sel1x` select inputs, as specified in the Functionality section above (look at the properties in the waveform viewer to see what the inputs are when the assertions failed).

1. Edit your assertions to account for the `sel1x` priority.
2. Rerun the simulation (remember to continue past the breakpoint).
3. Confirm that your new properties hold throughout the simulation.

Conclusions

One of the aims of this exercise is to teach you an important lesson on assertions. Assertions are ineffective if:

- ♦ The test data is incomplete, as in the first set of stimuli.
- ♦ The assertions are incorrectly written, as with our initial properties.

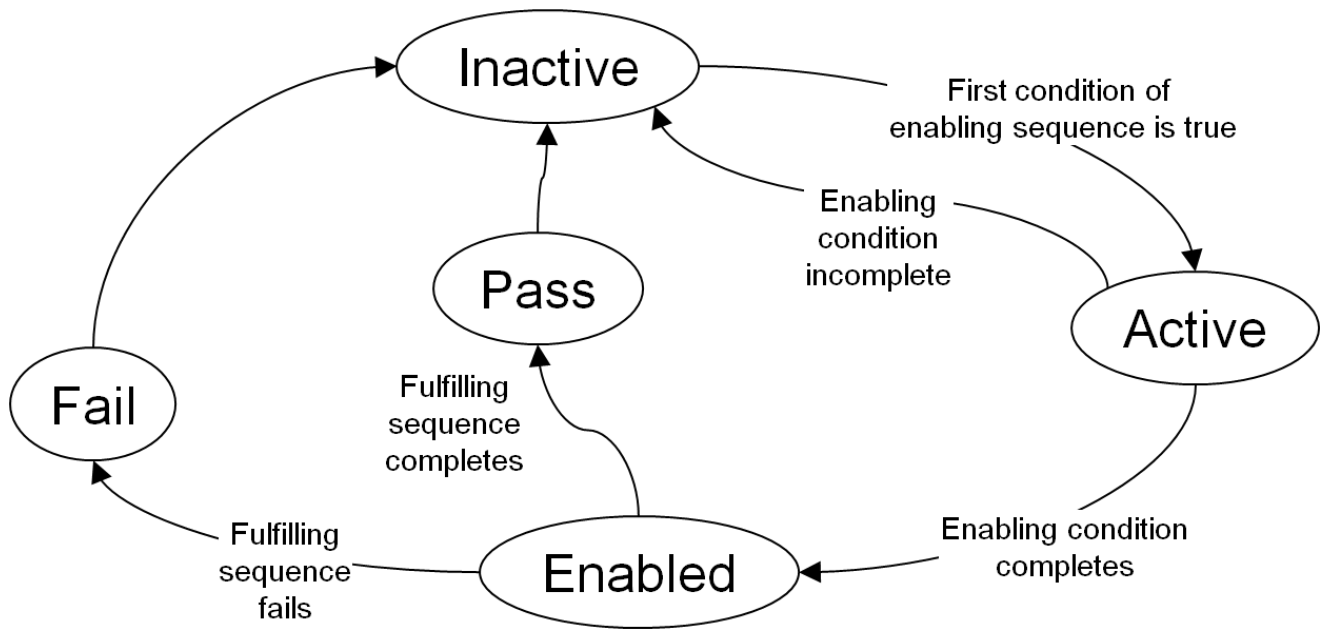


Lab 2 Sequence-Based Properties

Objective: To create and simulate sequence-based assertions.

Create a simple and a complex sequence-based assertion and simulate them with a supplied testbench to explore how your simulator handles and reports the various states in a multicyle assertion evaluation.

A multicyle sequence-based assertion can be in one of several states:



State	Description
Inactive	Waiting for the enabling sequence to begin
Active	Enabling sequence started and being evaluated
Enabled	Enabling sequence completed; evaluating fulfilling sequence
Pass	Fulfilling condition completed
Fail	Fulfilling condition failed

Creating the Simple Sequence Assertion

Working in the `lab03-svaseq` directory, edit the `seqtest.sv` file.

1. Create and assert a property `SIMPLE_SEQ`, clocked off the negative edge of `CLK`, to the following specification:
 - a. The enabling sequence is A at logic 1 followed immediately by B at logic 1 followed immediately by C at logic 1 (in consecutive samples).
 - b. The fulfilling sequence follows in the cycle after the enabling sequence completes and is J at logic 1 followed by K at logic 1 in consecutive samples.
 - c. The property is disabled if X is at logic 1 for any cycle.
2. Examine the rest of the testbench and make sure you understand its operation.
 Note that for each subtest, the testbench passes a character string to a subprogram `do_test`. The `;` character is the cycle delimiter. Other characters correspond to signals driven high by the subprogram during that clock cycle.

Leave the commented out section – this is for testing the complex sequence.

2. Simulate your design in GUI mode using the following `xrun` options:


```
xrun seqtest.sv -gui -access +rwc -linedebug
```
3. Set a breakpoint at the line which contains `breakpoint=breakpoint+1` in the task `do_test`. You can use this breakpoint to step through the subtests. The simulator will otherwise stop at only the failing tests.
4. Run the simulation. If you have successfully created the simple sequence assertion, you should see three assertion failures. Two of the failures occur within the same subtest.
5. Rerun the simulation up to the first failing subtest.

Question 1 – Which subtest failed and why did it fail?

.....

Hint: Use the waveform viewer to examine the signal in the sequence and the states of the assertion. Compare this failing test to previous passing tests.

6. Run the simulation up to the second failing subtest.

Question 2 – Which subtest failed and why did it fail?

.....

Creating the Complex Sequence Assertion

7. Add a complex sequence assertion `COMPLEX_SEQ`, clocked off the negative edge of `CLK`, to the following specification:
 - a. The enabling sequence is `C` at logic 1 followed immediately by between one and three samples, in which `B` is at logic 1 followed immediately by `A` at logic 1.
 - b. The fulfilling sequence follows in the cycle after the enabling sequence completes and is `J` at logic 1 for four samples, the last of which is followed immediately in the next sample by `K` at logic 1.
 - c. The assertion is disabled if `X` is at logic 1 for any cycle.

8. Comment out the stimulus for the simple sequence assertion and un-comment the stimulus for the complex assertion.

Compile `seqtest.sv`, fix any compilation errors and load into the simulator. You can set a breakpoint on the signal breakpoint to step through the subtests.

9. Run the stimulation. If you have successfully created the complex sequence assertion, you should see one failure when you simulate the design:
10. Run the simulation up to the failing test.

Question 3 – Which subtest failed and why did it fail?

.....

2. Add stimuli to explore and demonstrate answers to the following questions:

Question 4 – If a `K` occurs in the same cycle as the fourth `J`, e.g.:

```
C; B; B; A; J; J; J; JK; ;
```

When and why does the assertion fail?

.....

Question 5 – If multiple `As` occur in the same cycles as the second and third `Bs`, as seen in this example:

```
C; B; BA; BA; A; J; J; J; J; K;
```

How many assertions are enabled? How many fail? When and why do they fail?

.....

Question 6 – How can you modify the stimulus of Question 5 to ensure all assertions pass?

.....

Answers to Questions

Question 1 – Which test failed and why did it fail?

The 3rd subtest of the simple assertion fails to complete the fulfilling sequence – J is not immediately followed by K.

Question 2 – Which test failed and why did it fail?

The 5th subtest of the simple assertion fails to complete the fulfilling sequence for two overlapping traces of the sequence. Both traces fail because K does not immediately follow J. These two failures occur in consecutive cycles.

Question 3 – Which test failed and why did it fail?

The 6th subtest of the complex assertion fails to complete the fulfilling sequence – K does not immediately follow the fourth occurrence of J.

Question 4 – If a K occurs in the same cycle as the fourth J, when and why does the assertion fail?

The assertion fails on the cycle following the 4th occurrence of J. K is not checked on the fourth occurrence of J but is checked in the cycle immediately following. As K is not found, the assertion fails.

Question 5 – If multiple As occur in the same cycles as the second and third Bs, that is, from the stimulus:

C ; B ; BA ; BA ; A ; J ; J ; J ; J ; K ;

How many assertions are enabled? How many fail? When and why do they fail?

Three attempts start on cycle 1 and are enabled, at cycles 3, 4 and 5 of the above stimulus. The copy enabled on cycle 4 and cycle 3 fails because the first condition of the fulfilling sequence does not occur. The copy enabled on cycle 5 completes successfully.

Question 6 – How can you modify the stimulus of Question 5 to ensure all assertions pass?

The following stimulus passes all the assertions:

C ; B ; BA ; BAJ ; AJ ; J ; J ; JK ; JK ; K ;

